Operating Systems: Processes, Threads & CPU Scheduling Algorithms

Process Control Blocks (PCB), Context Switching Mechanics, Completely Fair Scheduler (CFS), and MLFQ

Doc Id: OS-SCHED-2024-001 | **Practice:** CPU Scheduling & Concurrency | **System:** Linux CFS, POSIX pthreads, Windows Kernel Executive | **Year:** 2024

1. Process Lifecycle and Process Control Block (PCB)

A process is an instance of a program in execution, encapsulating an address space, execution state, and system resources. In modern operating systems, the kernel manages processes through a primary kernel data structure termed the **Process Control Block (PCB)** (in Linux, struct task_struct).

The PCB stores all essential state needed to suspend and resume process execution:

• **Process Identifier (PID):** Unique integer identifying the process within the namespace.

• **Process State:** State transitions among NEW, READY, RUNNING, WAITING (blocked on I/O or event), and TERMINATED (zombie until reaped).

• **Program Counter (PC) & CPU Registers:** The instruction address and contents of data/pointer registers (RAX, RSP, RBP, CR3).

• **Memory Management Information:** Page table base pointers, segment tables, and virtual memory boundaries.

• **Accounting & Scheduling Information:** CPU time consumed, priority (nice value from -20 to +19), virtual runtime (vruntime), and scheduling class.

• **I/O Status & Open File Descriptors:** Array of pointers to open file table entries (stdin 0, stdout 1, stderr 2).

> *Context Switch Cost: A hardware context switch saves CPU registers to kernel stack, swaps CR3 (flushing non-global TLB entries unless PCID is supported), and loads new registers, incurring 1.2 to 3.5 microseconds of direct overhead plus subsequent cache cold-miss penalties.*

2. Threads vs. Processes: Architectural Trade-Offs

A thread is the smallest schedulable execution unit within a process. Multiple threads within the same process share code, global data, open file descriptors, and heap memory, but each thread maintains its own private Thread Control Block (TCB), Program Counter, stack frame, and register state.

Dimension	Process (fork)	Kernel Thread (pthread_create)	User-Level Green Thread / Coroutine
Address Space	Isolated (Private MMU mapping)	Shared with parent process	Shared within single thread
Creation Latency	High (copy-on-write page tables)	Moderate (~10-25 µs)	Ultra-low (~0.1-0.5 µs)
Context Switch Cost	1.5 - 3.8 µs (TLB flush / PCID reload)	0.4 - 1.1 µs (no MMU page table reload)	0.02 - 0.08 µs (user-space registers only)
Communication (IPC)	Pipes, Shared Memory, Sockets	Direct shared memory + mutexes	Channels or shared objects
OS Schedulability	Independently scheduled by kernel	Independently scheduled on cores	Multiplexed M:N on kernel threads

3. CPU Scheduling Algorithms: Comparative Evaluation

CPU scheduling determines which READY process receives the CPU core when the running task yields, blocks on I/O, or exceeds its timeslice quantum.

Key evaluation metrics include: **Turnaround Time** (completion minus arrival), **Waiting Time** (time spent in READY queue), **Response Time** (first CPU dispatch minus arrival), and **Throughput** (processes completed per unit time).

Algorithm	Preemptive?	Target Optimization	Turnaround Time	Risk / Failure Mode
First-Come First-Served (FCFS)	No	Simplicity & FIFO ordering	High (convoy effect)	Convoy effect on long CPU burst
Shortest Job First (SJF)	No	Minimum average wait time	Optimal (non-preemptive)	Starvation of long jobs, burst unpredictable
Shortest Remaining Time First (SRTF)	Yes	Minimum turnaround time	Optimal (preemptive)	High context switch overhead & starvation
Round Robin (RR)	Yes	Interactive response time	Moderate (depends on quantum)	Degrades to FCFS if quantum too large
Priority Scheduling	Both	Differentiated QoS classes	Varied by priority rank	Priority inversion and indefinite starvation
Multilevel Feedback Queue (MLFQ)	Yes	Separates I/O vs CPU bound	Near-optimal dynamic	Complex tuning of decay and boost intervals
Linux Completely Fair Scheduler (CFS)	Yes	Fair CPU time proportion	Proportional fair share	Slight scheduling latency jitter for micro-threads

4. Linux Completely Fair Scheduler (CFS) Mechanics

The Linux Completely Fair Scheduler (CFS) does not use fixed timeslice quantums. Instead, it maintains a red-black tree indexed by each task's **virtual runtime (vruntime)**.

Virtual runtime tracks the normalized amount of CPU time a process has executed:

• vruntime += delta_exec * (NICE_0_LOAD / task_weight)

• A task with nice value 0 has weight 1024. A task with nice -5 has weight 3121 (its vruntime advances slower, giving it ~3x more physical CPU time).

• The kernel always selects the leftmost node in the red-black tree (minimum vruntime) for the next CPU dispatch, ensuring proportional fairness and self-balancing scheduling in O(log N) insertion and O(1) lookup.